

CREATE: A Closed-Loop Reward Engineering Agent with Training Evidence for Reinforcement Learning

Anonymous Authors
Anonymous Submission

Abstract—Reward engineering for reinforcement learning remains iterative and labor-intensive: large language models (LLMs) can generate reward programs, but a generated reward is only meaningfully evaluated after an expensive policy-training run, and most pipelines discard the resulting training evidence. This paper introduces CREATE, a closed-loop reward engineering agent that treats each completed training run as a diagnostic observation rather than a scalar outcome. During repair, CREATE uses tool calls to inspect component behavior, diagnose which reward semantic failed, select a bounded single-semantic action, and record the evidence–action–outcome chain in persistent memory while a best archive prevents regression. On LunarLander-v3, CREATE solves all five lineages under a ten-evaluation budget, whereas budget-matched independent generation and baseline solve none. Mechanism ablations confirm that tool-mediated evidence and bounded reward actions are coupled requirements: neither alone enables reliable repair. A secondary study on BipedalWalker-v3 supports transfer to continuous-action locomotion. These results establish reward repair as an agentic closed-loop process in which failed training becomes actionable evidence.

Index Terms—reinforcement learning, reward design, large language models, language agents, training evidence

I. INTRODUCTION

Reinforcement learning (RL) depends critically on reward functions that express task intent while providing informative feedback for policy optimization [1]. Reward shaping can improve credit assignment, but only restricted transformations are guaranteed to preserve the optimal policy [2]. Inverse reward design further shows that a written reward should be treated as imperfect evidence of designer intent rather than as the intent itself [3]. Reward misspecification can therefore produce unsafe or unintended behavior even when the specified signal appears semantically reasonable [4].

Reward engineering is consequently an iterative debugging process rather than a one-shot coding task. Each completed training run produces more than a scalar return: episode length, termination patterns, reward-component activation, magnitude balance, and temporal trends can reveal how the reward program shaped the learned policy. Converting these heterogeneous records into a precise reward modification, however, still requires substantial human analysis.

Code-capable large language models (LLMs) can generate reward programs from task descriptions [5], [6] and iteratively refine them through training feedback and evolutionary search [7]–[10]. Diagnostic-driven refinement further treats reward failure as a debugging problem [11]. In parallel, LLMs

are increasingly organized as agents that observe, reason, and act across trials, using stored feedback to improve later decisions [12]–[14]. These developments motivate an agent-oriented view of reward engineering in which policy training supplies observations and reward-code revision constitutes an external action.

This paper explores an agent-oriented view of reward engineering through CREATE, a *Closed-Loop Reward Engineering Agent with Training Evidence*. CREATE places a reward-engineering agent above an inner RL policy learner. After each policy-training run, the reward-engineering agent uses explicit tool calls to inspect training feedback, reward and lineage history, and component-level trends while it reasons about the next repair. It then diagnoses one principal semantic failure and applies a localized edit: L1 tunes parameters, L2 refactors one semantic, and L3 redesigns the reward structure after repeated local failure. Intervention memory retains this evidence chain across rounds, while a separate best archive protects the strongest solution from later exploratory regressions.

Experiments on LunarLander-v3 compare CREATE against budget-matched independent generation and baseline, with two mechanism ablations isolating evidence enrichment and hierarchical semantic editing. BipedalWalker-v3 provides a secondary cross-task validation. Under a ten-evaluation budget, the experiments demonstrate that structured observation, persistent state, and bounded reward actions convert failed policy training into reliable reward repair.

II. RELATED WORK

A. Reward Engineering for Reinforcement Learning

Reward shaping augments task objectives with intermediate signals that improve credit assignment while preserving the optimal policy only under restricted transformations [2]. Inverse reward design treats a specified reward as imperfect evidence of designer intent and highlights the gap between written objectives and desired behavior [3]. More broadly, reward misspecification can produce unsafe or unintended behavior even when the reward appears semantically reasonable [4]. These results motivate methods that treat reward engineering as an iterative debugging problem rather than a one-shot specification step.

B. LLM-Based Reward Generation and Refinement

LLMs can map task descriptions to executable reward code. Language to Rewards demonstrates language-conditioned

reward generation for robotic skill synthesis [5]. Text2Reward frames reward shaping as interpretable program synthesis from language instructions [6]. Eureka combines reward-code generation with evolutionary optimization and component-level feedback [7]. CARD incorporates dynamic feedback into LLM-driven reward design [8]. REvolve studies reward evolution with human feedback in the loop [9]. A chain-of-thought reward-engineering approach shows that explicit reasoning traces can improve reward-design transparency [10]. Diagnostic-driven refinement treats reward failure as a debugging problem and uses training diagnostics for targeted revision [11]. Collectively, these methods show that training feedback supports reward revision beyond one-shot generation.

C. LLM Agents and Agentic Reward Engineering

Language-agent research organizes LLMs to maintain state and act in external processes. ReAct couples reasoning with tool-mediated actions [12]. Reflexion stores verbal feedback as episodic experience for later decisions [13]. Self-Refine shows how iterative self-feedback can improve generated outputs without parameter updates [14]. These mechanisms are entering reward design: RF-Agent formulates reward construction as sequential decision making [15], and STRIDE applies agentic reward engineering to humanoid locomotion [16]. CREATE follows this direction but centers agency on instrumented policy training: the reward-engineering agent calls tools over training records, reasons over the returned evidence, and commits to a semantically localized action measured by the next complete training run.

III. METHODOLOGY

CREATE places a reward-engineering agent above an underlying RL learner. Each round trains a fresh policy on the current generated reward, evaluates it with the native task objective, and returns the completed run to the reward-engineering agent. As shown in Fig. 1, the agent issues tool calls while reasoning, reads the returned evidence, and then commits to a validated reward edit.

The resulting observe–reflect–act loop is written as

$$\begin{aligned} O_t &= \Phi_{\mathcal{T}}(\mathcal{D}_t), \\ (H_t, A_t) &= \Psi(R_t, O_t, M_t, B_t), \\ R_{t+1} &= \mathcal{U}(R_t, A_t), \end{aligned} \quad (1)$$

where $\Phi_{\mathcal{T}}$ denotes the multi-call tool-use interface and O_t is evidence returned during reasoning. The reward-engineering agent combines O_t , the current reward R_t , intervention memory M_t , and best archive B_t to produce reflection H_t , reward action A_t , and the next executable reward program R_{t+1} .

A. Task Understanding and Reward Initialization

Given the task description, environment interface, and reward-code constraints, CREATE instantiates named components for behavior semantics such as task progress, stability, target events, and action cost. The initial reward is

$$R_0(s, a, s') = \sum_{i=1}^{m_0} w_i^{(0)} r_i^{(0)}(s, a, s'), \quad (2)$$

Named components are logged independently during training, enabling component-level diagnosis in later rounds. Generated code is checked for interface and execution validity before policy training; prompts and validation rules are provided in the appendix.

B. Training–Evidence–Driven Observe–Reflect–Act

At iteration t , the underlying RL agent optimizes the generated reward, while CREATE evaluates the learned policy with the environment-native objective:

$$\begin{aligned} \pi_t &= \arg \max_{\pi} \mathbb{E}_{\tau \sim \pi} \left[\sum_{k=0}^{H-1} \gamma^k R_t(s_k, a_k, s_{k+1}) \right], \\ J_t &= \frac{1}{N} \sum_{n=1}^N \sum_{k=0}^{H_n-1} r_{\text{env}}(s_k^{(n)}, a_k^{(n)}, s_{k+1}^{(n)}). \end{aligned} \quad (3)$$

Thus, the generated reward shapes policy learning, whereas the native objective measures task completion. Each run logs episode outcomes, termination patterns, component activation and scale, temporal trends, and reward-version deltas. During reasoning, the reward-engineering agent queries these records through tools for lineage memory, training feedback, reward code, prior reflections, component history, environment context, and checkpoint trends. The returned observations support diagnosis, edit-level selection, and reward-code modification.

Component statistics serve as cues rather than editing rules: inactivity suggests unreachable gates or sparse terms, excessive activation and magnitude suggest dominant proxies, and frequent activation with weak contribution suggests poor scale or mathematical form. The agent combines these cues with policy outcomes, source code, and prior interventions to identify one principal reward semantic:

$$A_t = (q_t, \ell_t, \Delta_t), \quad \text{supp}(\Delta_t) \subseteq \mathcal{C}(q_t), \quad (4)$$

where q_t is the selected semantic, ℓ_t the edit level, and $\mathcal{C}(q_t)$ the components and parameters implementing that semantic. Related components may be edited jointly only when they express the same design intent. L1 calibrates coefficients, thresholds, or gates; L2 adds, removes, or refactors components while preserving the target semantic; and L3 replans the reward structure after repeated local failure.

C. Intervention Memory and Best-Reward Selection

After each round, CREATE records the observation, diagnosed semantic, reward action, expected change, and subsequent outcome in persistent intervention memory. This history supports cross-round comparison and discourages repeated failed edits. The best archive is maintained separately from the active lineage to prevent later regressions from overwriting a previously discovered solution. After at most K reward evaluations, CREATE returns

$$t^* = \arg \max_{0 \leq t < K} J_t, \quad R^* = R_{t^*}. \quad (5)$$

The selected reward–policy pair is subsequently evaluated on independent episode seeds to assess stability. Together, tool-use observation, bounded semantic editing, intervention memory,

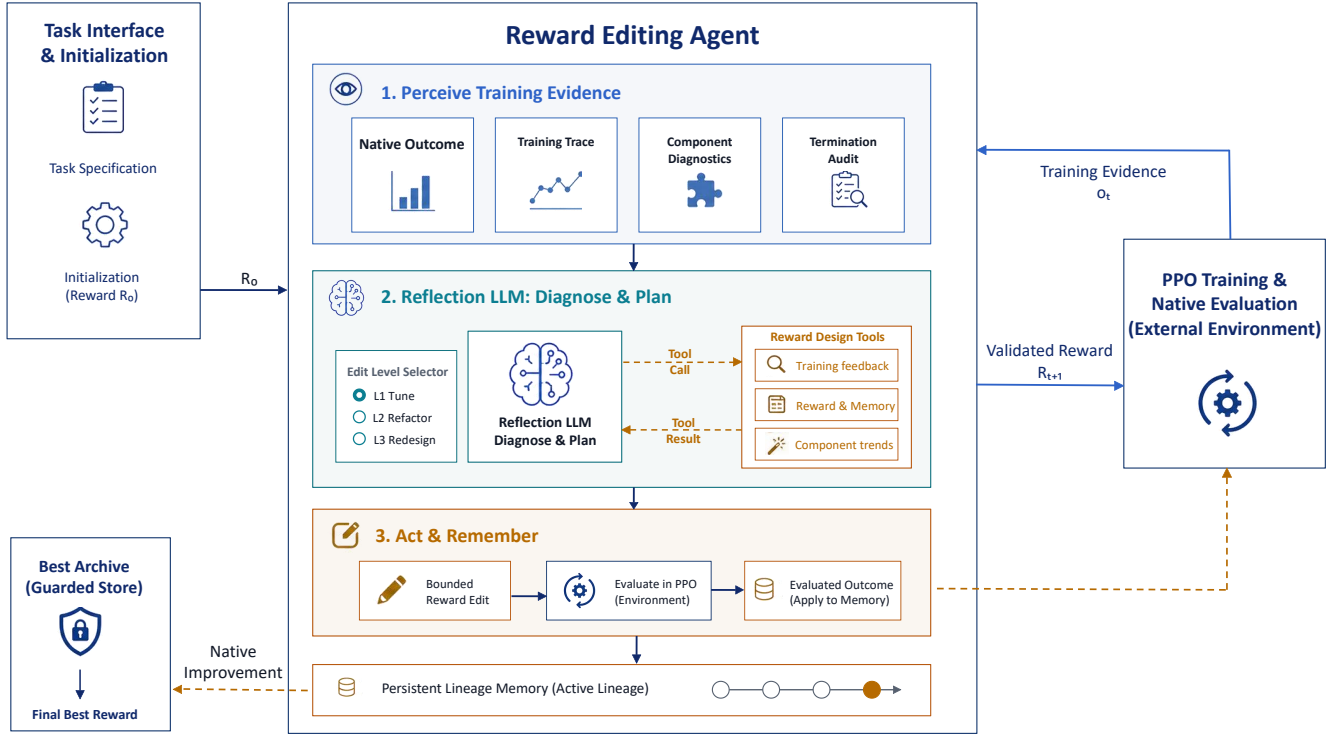


Fig. 1. Overview of CREATE. The reward-engineering agent uses tool calls to inspect training feedback, reward code, lineage memory, and component trends before selecting a localized reward edit. Persistent memory records intervention outcomes, and the best archive retains the reward program with the highest native-task performance.

and best-reward selection turn each completed policy-training run into a diagnostic observation and each subsequent edit into a localized test of one reward semantic.

IV. EXPERIMENTAL PROCEDURE

A. Experimental Setup and Configuration

Experiments use LunarLander-v3 and BipedalWalker-v3 from Gymnasium [17]. LunarLander is the primary benchmark for baseline comparison and ablation; BipedalWalker validates continuous-action locomotion. Every reward program trains a fresh PPO policy [18] through Stable-Baselines3 [19]. LunarLander uses 10^6 environment steps per candidate and BipedalWalker uses 5×10^6 . DeepSeek V4 Pro is the LLM backbone for all LunarLander conditions; prompts, settings, and hyperparameters are provided in the appendix.

B. Condition Definitions

Five conditions are compared on LunarLander-v3 under a shared LLM backbone, PPO budget, reward interface, and code-check pipeline. All conditions start from the same task description and environment source/interface with the reward-function portion removed from the input, giving them the same task-understanding start. Each condition contains five independent reward-search lineages and at most ten complete reward evaluations. The iterative conditions use the same initialization protocol from Section III-A, but each lineage begins from its own generated first reward. Their first-reward evaluation is -2.21 ± 73.38 with no solved lineage, confirming

that plausible generated rewards are not reliably learnable without refinement.

Independent-10 serves as a budget-matched search-breadth control: it generates ten unrelated initial reward candidates, each from an independent invocation of the reward-initialization phase, and evaluates each with a separate policy-training run. The best candidate is selected post hoc. This condition tests whether evaluating ten unrelated candidates, without any cross-round information flow, can match CREATE’s iterative diagnostic process.

Baseline is the stateless sequential-revision control. It retains sequential LLM invocation but removes the tool-use interface, intervention memory, cross-round comparison, and hierarchical semantic editing. The LLM is limited to basic current-round feedback and may modify any number of components simultaneously, with no constraint on edit scope. It revises rewards based on previous outcomes and retains the best-so-far solution, but operates without memory, tool-returned evidence, or bounded reward-action constraints.

CREATE without evidence enrichment retains memory, best archive, and bounded semantic editing, but disables tool calls over training records, reward code, lineage history, and component trends. The reward-engineering agent can consult only aggregate current-round feedback, so it lacks the component-level and cross-round observations that CREATE obtains through tool use.

CREATE without hierarchical editing retains the same tool-use observations, memory, and best archive as CREATE,

TABLE I
SEED-LEVEL LUNARLANDER BEST-SEARCH FITNESS.

Condition	s0	s1	s2	s3	s4	Mean
<i>Budget-matched baselines</i>						
Independent-10	-13.1	170.8	55.1	-106.7	-109.7	-0.7 ± 118.1
Baseline	25.4	-5.0	-10.1	151.3	58.9	44.1 ± 66.0
CREATE	224.2	240.6	220.2	253.7	206.1	229.0 ± 18.5
<i>Mechanism ablations</i>						
w/o evidence	239.5	170.4	-110.1	115.5	259.5	135.0 ± 148.4
w/o hierarchy	169.9	130.6	71.1	59.2	140.3	114.2 ± 47.3
CREATE	224.2	240.6	220.2	253.7	206.1	229.0 ± 18.5

but removes the semantic-localization constraint defined in Eq. 4. The agent may therefore rewrite several unrelated reward components in one round.

CREATE runs the complete architecture described in Section III.

C. Training and Evaluation Protocol

Each reward-design round consists of a complete PPO training run followed by native evaluation. The policy is optimized with the generated reward R_t , whereas Eq. 3 measures candidate quality only by the untouched environment reward r_{env} . We define *search fitness* as the mean undiscounted native episodic return over 20 fixed evaluation episodes and *best search fitness* as the largest value retained within a lineage. After reward selection, the returned policy is evaluated over 100 unseen episodes; this is reported as *test fitness*. The task-solving thresholds are 200 for LunarLander and 300 for BipedalWalker.

Between rounds, the reward-engineering agent uses the tool interface $\Phi_{\mathcal{T}}$ in Eq. 1 to inspect selected training records, reward code, lineage memory, and component trends. It then selects one principal semantic and chooses an L1 calibration, L2 refactor, or L3 redesign action under Eq. 4. The modified program is validated before the next round. Eq. 5 updates the best archive only when a new candidate exceeds the current incumbent in native search fitness. On BipedalWalker-v3, the complete CREATE workflow is evaluated for cross-task validation.

V. RESULTS AND ANALYSIS

A. Reward Search Under a Matched Training Budget

Search breadth alone does not produce reliable reward repair. Table I reports per-seed best search fitness under a ten-evaluation budget, with bold entries crossing the 200 threshold. Independent-10 evaluates ten unrelated candidates without cross-round information flow, yet solves no lineage—ruling out that candidate count alone drives improvement. The iterative conditions share the same task facts, environment interface, reward-generation protocol, and PPO budget; subsequent differences therefore reflect each architecture’s capacity to convert training outcomes into better reward functions.

Iterative rewriting without tool-use evidence or memory produces non-monotonic progress. Baseline retains sequential LLM invocation but removes tool access, memory, and editing constraints. It solves no lineage, and in three seeds the initial reward outperforms every subsequent revision. Without component-level diagnostics or memory, the LLM cannot

distinguish a poorly calibrated coefficient from a fundamentally wrong semantic—each rewrite is effectively a guess. **CREATE**, by contrast, solves every lineage, demonstrating that agentic evidence seeking, not repeated invocation, drives reliable repair.

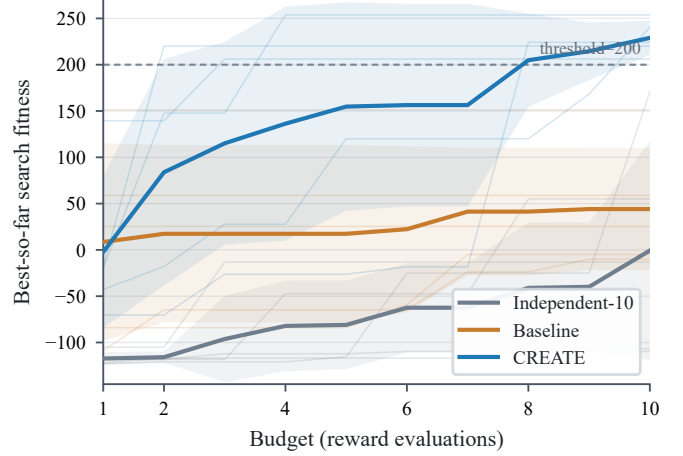


Fig. 2. Best-so-far LunarLander search fitness under a matched policy-training budget. Each curve averages five lineages after retaining the incumbent within each lineage; the dashed line denotes the task-solving threshold.

Fig. 2 reveals the qualitative gap: Independent-10 improves by chance, Baseline plateaus, and CREATE progressively raises its incumbent. The distinction is whether each training run becomes diagnostic evidence rather than a binary outcome. Fig. 3(a) shows that the same gap appears at the per-lineage level.

B. Mechanism Ablations

We now isolate *why* the closed loop works by testing two mechanism hypotheses from Section III.

Tool use enables agentic diagnosis. Activation rates, magnitude shares, and temporal trends let the reward-engineering agent distinguish mechanistically distinct failure modes—for instance, an inactive component could be unreachable, too weak, or behaviorally irrelevant, and a scalar sum cannot tell them apart.

Without tool-use access, the reward-engineering agent can consult only scalar current-round feedback. Table I shows that this variant solves only two of five lineages, with high dispersion and one lineage that remains negative throughout. Aggregate scores can support occasional repair but cannot distinguish *why* a component failed: *what* evidence the agent can inspect determines *which* diagnoses it can form.

Bounded reward actions enable single-trial attribution. Eq. 4 formalizes the semantic-localization constraint that makes each retraining run a controlled test of one intervention. This matters because reward revision is not ordinary program repair: an LLM can analyze code and diagnostics, but the policy consequence of changing reward weights, gates, or mathematical forms appears only after black-box RL training [11].

Without bounded actions, the agent retains tool-returned evidence and memory but may issue unconstrained multi-component rewrites. Table I shows that this variant solves no lineage, while Fig. 3(b) shows the same failure pattern visually.

If several components are returned or refactored together, a failed next run gives no stable diagnostic target: the agent may react by changing other components, coupling unrelated hypotheses and severing the evidence–action–outcome chain. The ablation therefore degenerates into metrics-conditioned trial and error, not experience-guided diagnosis; each 10^6 -step run must stay tied to one falsifiable reward hypothesis.

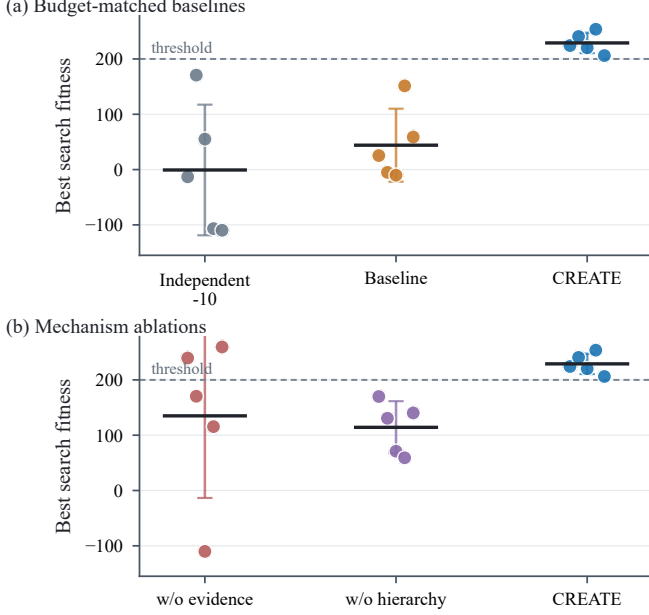


Fig. 3. LunarLander per-lineage search outcomes. (a) Independent-10, Baseline, and CREATE under a matched budget. (b) CREATE compared with the two mechanism ablations. Dots are lineages, black bars are means, whiskers denote standard deviations, and the dashed line denotes the task threshold.

Fig. 3(b) shows that tool-mediated evidence and bounded actions are coupled requirements. Evidence-poor repair is initialization-dependent, and unconstrained rewriting remains below threshold in every lineage. Full CREATE combines 5/5 solutions with a tight distribution, aligning high success rate with reliable attribution.

C. Agentic Diagnosis from Tool-Returned Evidence

We examine *how* the reward-engineering agent turns tool-returned evidence into a diagnosis and intervention through the most representative seed-0 trajectory in Fig. 4. This lineage contains both collapse and recovery; complete traces and additional component-evidence heatmaps are provided in the appendix.

Diagnostic trajectory. Subfigures b and c separate three kinds of components: guidance signals such as safe, approach, and landing proximity; safety or cost terms such as descent and fuel; and landing-quality events such as touchdown and stable. Iterations 1–5 fail to produce stable landing, with the best score still below zero. Before iteration 6, the agent observes from the training records that distance progress is active while episodes terminate early; the queried reward-design tools map this pattern to proxy misalignment and missing safe-descent feedback. The selected L2 action removes raw distance reduction and introduces state-based safe proximity, descent

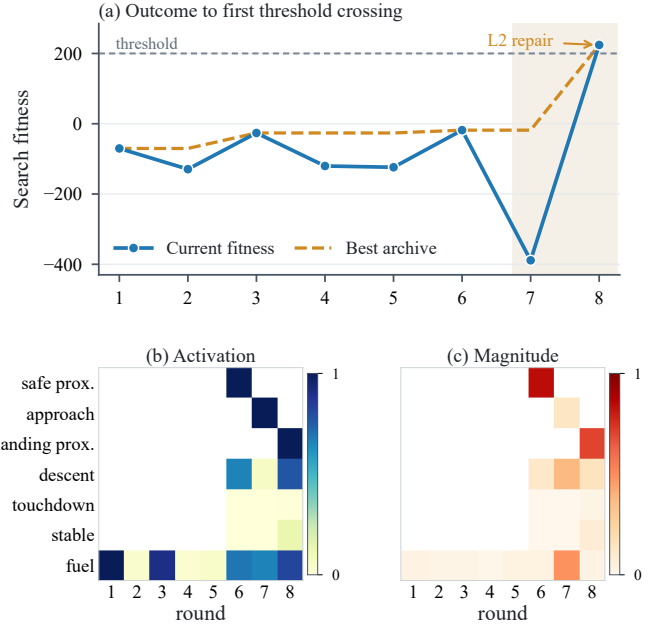


Fig. 4. Auditable CREATE repair for LunarLander seed 0. (a) Current fitness and the protected best archive. (b) Component activation rates across rounds. (c) Component magnitude shares used during diagnosis.

safety, touchdown, and stable-landing components, visible as new rows in subfigure b. Iteration 6 improves to -18.3 and survives much longer, giving memory a useful but still unsolved reference point.

Collapse and recovery. The next reward version changes the approach signal back to a delta-style shaping term and collapses at iteration 7 to -388.7 . The tools then expose the failure signature reflected in subfigures b and c: touchdown and stable-landing components are not activated, approach shaping contributes negatively, and descent safety appears only rarely but with large penalty share. Rather than discarding the entire reward, the agent compares against iteration 6 and selects an L2 repair: replace the unstable delta signal with a bounded landing-proximity term that combines horizontal centering with a height factor, while keeping safety and landing components. Iteration 8 reaches 224.2 , and subfigure a shows the best archive preserving this recovered solution through later exploratory regression.

Mechanism summary. The case illustrates the full CREATE loop: tool calls inspect lineage state, including reward code, prior diagnoses, component activation, magnitude balance, and previous outcome; memory makes iteration 6 the recovery anchor and iteration 7 negative evidence; and the agent returns to the best preserved reward family. It then changes only the unstable semantic, so recovery is diagnostic rather than a fresh regeneration attempt.

D. Cross-Task Validation

BipedalWalker-v3 validates CREATE on continuous-action locomotion as a secondary domain with a different action space and gait-control reward structure. The ten-evaluation cap and input protocol remain unchanged, but the reward semantics shift from landing to forward gait and stability. Table II reports

the first reward version that crosses the task threshold, where v denotes the evaluated version.

TABLE II
BIPEDALWALKER-V3 CREATE SEARCH RESULTS.

Seed	Initial fitness	First $v \geq 300$	Best search
0	270.7	v_2	320.0
1	280.5	v_2	313.7
2	272.1	v_2	307.9
3	290.0	v_2	311.1
4	103.0	v_3	304.9
Mean	243.26 ± 70.45	2.2 versions	311.54 ± 5.17

All five BipedalWalker lineages cross the threshold within three evaluated reward versions, and the selected policies obtain 310.82 ± 5.03 test fitness over unseen episodes. Four lineages begin near the threshold and mainly require calibration; seed 4 starts from 103.0 and reaches 304.9 after one additional repair round. This secondary result summarizes the cross-task evidence: the same observe–remember–act workflow remains operational when reward repair shifts from discrete landing semantics to continuous gait control.

VI. CONCLUSION

This study presented CREATE, a closed-loop reward engineering agent that turns completed policy-training runs into diagnostic observations. On LunarLander-v3, CREATE solved all five lineages under a ten-evaluation budget, while budget-matched independent generation and baseline solved none. Ablations show that tool-use observation and bounded reward actions must work together, and BipedalWalker-v3 indicates that the observe–remember–act loop can extend beyond discrete landing. These results position diagnostic reward repair as a practical path for agentic reward engineering. Future work can make the loop more perceptive and efficient: monitor training convergence to terminate unpromising reward programs early, add visual rollout evidence so the agent can inspect behavioral failure modes, and connect the same repair loop to stronger policy optimizers and harder robotic tasks.

REFERENCES

- [1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [2] A. Y. Ng, D. Harada, and S. Russell, “Policy invariance under reward transformations: Theory and application to reward shaping,” in *Proceedings of the 16th International Conference on Machine Learning*, 1999, pp. 278–287.
- [3] D. Hadfield-Menell, S. Milli, P. Abbeel, S. Russell, and A. Dragan, “Inverse reward design,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [4] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, “Concrete problems in AI safety,” *arXiv preprint arXiv:1606.06565*, 2016.
- [5] W. Yu, N. Gileadi, C. Fu, S. Kirmani, K.-H. Lee, M. G. Arenas, H.-T. L. Chiang, T. Erez, L. Hasenclever, J. Humplik, B. Ichter, T. Xiao, P. Xu, A. Zeng, T. Zhang, N. Heess, D. Sadigh, J. Tan, Y. Tassa, and F. Xia, “Language to rewards for robotic skill synthesis,” in *Proceedings of the 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229, 2023, pp. 374–404.
- [6] T. Xie, S. Zhao, C. H. Wu, Y. Liu, Q. Luo, V. Zhong, Y. Yang, and T. Yu, “Text2reward: Reward shaping with language models for reinforcement learning,” in *International Conference on Learning Representations*, 2024.

- [7] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, J. Fan, and A. Anandkumar, “Eureka: Human-level reward design via coding large language models,” in *International Conference on Learning Representations*, 2024.
- [8] S. Sun, R. Liu, J. Lyu, J.-W. Yang, L. Zhang, and X. Li, “A large language model-driven reward design framework via dynamic feedback for reinforcement learning,” *Knowledge-Based Systems*, vol. 326, p. 114065, 2025.
- [9] R. Hazra, A. Sygkounas, A. Persson, A. Loutfi, and P. Z. D. Martires, “REvolve: Reward evolution with large language models using human feedback,” in *International Conference on Learning Representations*, 2025.
- [10] X. Zhu, J. Du, Q. Fu, and L. Chen, “LLM-based reward engineering for reinforcement learning: A chain of thought approach,” in *2025 10th International Conference on Cloud Computing and Big Data Analytics (ICCCBDA)*, 2025, pp. 222–227.
- [11] Y. Wang, Y. Tang, B. Liu, X. Liu, and D. Shang, “When LLM reward design fails: Diagnostic-driven refinement for sparse structured RL,” *arXiv preprint arXiv:2605.28918*, 2026.
- [12] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations*, 2023.
- [13] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [14] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhunoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, “Self-refine: Iterative refinement with self-feedback,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [15] N. Gao, X. Zhang, X. Jiang, M. You, M. Zhang, and Y. Deng, “RF-Agent: Automated reward function design via language agent tree search,” in *Advances in Neural Information Processing Systems*, vol. 38, 2025.
- [16] Z. Wu, J. Lu, Y. Chen, Y. Liu, Y. Zhuang, and L. Hu, “STRIDE: Automating reward design, deep reinforcement learning training and feedback optimization in humanoid robotics locomotion,” *arXiv preprint arXiv:2502.04692*, 2025.
- [17] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, Arjun KG, R. Perez-Vicente, A. Pierré, S. Schulhoff, J. J. Tai, H. Tan, and O. G. Younis, “Gymnasium: A standard interface for reinforcement learning environments,” *arXiv preprint arXiv:2407.17032*, 2024.
- [18] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [19] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.